

MÓDULO 1 · PIPELINE DE DATOS Y PREPARACIÓN PROFESIONAL

Limpieza e Imputación de Valores Faltantes

Material teórico — Calidad de datos, imputación y estrategias profesionales

Objetivo del material

Comprender cómo identificar, analizar y tratar valores faltantes en un conjunto de datos, seleccionando estrategias de imputación adecuadas y evitando problemas como la pérdida innecesaria de información y el data leakage.

1. La calidad de los datos

Un modelo de Machine Learning aprende a partir de los datos que recibe. Si esos datos contienen errores, inconsistencias o información ausente, el modelo puede aprender patrones incorrectos.

Por este motivo, la preparación de datos constituye una etapa fundamental de cualquier proyecto profesional de Machine Learning.

- Datos incompletos.
- Datos duplicados.
- Valores incorrectos.
- Formatos inconsistentes.
- Valores extremos.
- Categorías mal escritas.
- Variables con unidades diferentes.

2. ¿Qué es un valor faltante?

Un **valor faltante** es una observación en la que no se dispone del valor correspondiente a una determinada variable.

En Pandas, los valores faltantes suelen representarse mediante **NaN** (Not a Number) o **None**, dependiendo del tipo de dato y de cómo se hayan cargado los datos.

Situación	Ejemplo	Problema
Celda vacía	Edad = NaN	No existe un valor registrado.
Valor especial	Edad = -1	Puede representar un faltante codificado incorrectamente.
Texto indicador	Ciudad = 'Desconocido'	El faltante fue almacenado como texto.
Código especial	9999	El código puede confundirse con un valor real.

3. ¿Por qué aparecen valores faltantes?

- El usuario no proporcionó un dato.
- Un sensor dejó de funcionar.

- Una pregunta no era aplicable.
- Se produjo un error durante la carga.
- Los datos provienen de diferentes fuentes.
- El proceso de integración perdió información.
- El valor fue ocultado por motivos de privacidad.

La causa es importante porque condiciona la estrategia de tratamiento.

4. Detectar valores faltantes con Pandas

La primera tarea es identificar cuántos valores faltantes existen y en qué variables aparecen.

```
df.isnull()
df.isnull().sum()
```

`isnull()` devuelve una estructura booleana que indica qué posiciones contienen valores faltantes.

`isnull().sum()` permite obtener la cantidad de valores faltantes por columna.

5. Porcentaje de valores faltantes

La cantidad absoluta de valores faltantes no siempre es suficiente. También debemos conocer qué proporción representan.

```
porcentaje_faltantes = df.isnull().mean() * 100
print(porcentaje_faltantes)
```

Por ejemplo, 20 valores faltantes pueden ser insignificantes en un dataset de un millón de registros, pero pueden ser muy importantes en un dataset de 50 registros.

6. Analizar el patrón de ausencia

No todos los valores faltantes aparecen de la misma manera. Es importante observar si la ausencia está concentrada en determinadas variables, grupos o períodos.

- ¿Los faltantes aparecen en una única columna?
- ¿Se concentran en determinados registros?
- ¿Aparecen más en un grupo de clientes?
- ¿Están relacionados con una determinada fecha?
- ¿Existe alguna variable que permita explicar su ausencia?

7. Eliminar filas con valores faltantes

Una estrategia sencilla consiste en eliminar las filas que contienen valores faltantes.

```
df_limpio = df.dropna()
```

Esta estrategia puede ser apropiada cuando existen pocos valores faltantes y su eliminación no implica perder información importante.

Riesgo: si existen muchos valores faltantes, podemos terminar eliminando una gran cantidad de registros.

8. Eliminar columnas

Si una variable contiene una proporción extremadamente alta de valores faltantes y no resulta útil para el problema, puede considerarse eliminarla.

```
df = df.drop(columns=['variable'])
```

Sin embargo, esta decisión debe basarse en el contexto del problema y no solamente en un porcentaje arbitrario.

9. Imputación de valores faltantes

La **imputación** consiste en reemplazar un valor faltante por una estimación.

El objetivo es conservar la mayor cantidad posible de información sin introducir distorsiones importantes.

Estrategia	Cuándo utilizarla	Ventaja / riesgo
Media	Variables numéricas con distribución relativamente estable.	Simple, pero sensible a valores extremos.
Mediana	Variables numéricas con asimetría o outliers.	Más robusta frente a valores extremos.
Moda	Variables categóricas.	Simple, pero puede aumentar artificialmente una categoría.
Por grupos	Cuando existen grupos con comportamientos diferentes.	Más contextualizada, pero requiere mayor análisis.
KNN	Cuando la similitud entre observaciones aporta información.	Puede ser costosa y requiere variables adecuadamente escaladas.
Iterativa	Cuando las variables presentan relaciones útiles para estimar faltantes.	Más flexible, pero más compleja y costosa.

10. Imputación mediante la media

La media puede utilizarse para completar valores faltantes de una variable numérica.

```
media = df['edad'].mean()
df['edad'] = df['edad'].fillna(media)
```

Es sencilla, pero puede verse afectada por valores extremos.

11. Imputación mediante la mediana

La mediana suele ser una alternativa más robusta cuando la variable presenta asimetría o valores atípicos.

```
mediana = df['ingresos'].median()
df['ingresos'] = df['ingresos'].fillna(mediana)
```

La mediana divide los valores ordenados en dos grupos de igual tamaño y suele verse menos afectada por valores extremos.

12. Imputación mediante la moda

La moda es el valor que aparece con mayor frecuencia. Es especialmente útil para variables categóricas.

```
moda = df['plan'].mode()[0]
df['plan'] = df['plan'].fillna(moda)
```

Debe utilizarse con cuidado porque puede aumentar artificialmente la frecuencia de una categoría.

13. Imputación por grupos

En algunos problemas, la media o mediana global no representa bien a todos los registros. Puede ser más apropiado calcular la estadística dentro de un grupo.

Por ejemplo, los ingresos pueden comportarse de manera diferente según el tipo de cliente.

```
df['ingresos'] = df.groupby('tipo_cliente')['ingresos'].transform(
    lambda x: x.fillna(x.median())
)
```

Esta estrategia incorpora información contextual y puede ser más adecuada que una única mediana global.

14. SimpleImputer de Scikit-learn

Scikit-learn ofrece herramientas específicas para realizar imputación dentro de pipelines de Machine Learning.

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='median')
X_imputado = imputer.fit_transform(X)
```

Entre las estrategias disponibles se encuentran **mean**, **median** y **most_frequent**, entre otras.

15. ¿Por qué utilizar Scikit-learn?

- Permite integrar la imputación en un pipeline.
- Facilita reproducir el mismo procedimiento.
- Evita aplicar manualmente transformaciones diferentes.
- Permite separar el aprendizaje de parámetros y la transformación.
- Se integra con otros pasos de preparación y modelado.

16. KNNImputer

KNNImputer utiliza observaciones similares para estimar los valores faltantes.

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)
X_imputado = imputer.fit_transform(X)
```

La idea es que observaciones similares pueden proporcionar información útil para estimar el valor que falta.

Su principal desventaja es que puede resultar más costoso computacionalmente y requiere especial atención a las escalas de las variables.

17. Imputación iterativa

La imputación iterativa utiliza las relaciones entre variables para estimar progresivamente los valores faltantes.

Una implementación disponible en Scikit-learn es **IterativeImputer**.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

imputer = IterativeImputer(random_state=42)
X_imputado = imputer.fit_transform(X)
```

Este método puede producir estimaciones más sofisticadas, pero también requiere mayor complejidad y análisis.

18. Comparación de estrategias

Método	Complejidad	Conserva relaciones	Uso habitual
Media	Baja	No	Datos numéricos simples
Mediana	Baja	No	Datos con outliers
Moda	Baja	No	Variables categóricas
KNN	Media	Parcialmente	Datos donde la similitud importa
Iterativa	Alta	Sí	Relaciones entre variables

19. ¿Cómo elegir una estrategia?

No existe una estrategia universalmente correcta. La elección depende de la naturaleza de los datos, la cantidad de faltantes y el problema que se intenta resolver.

- Cantidad de valores faltantes.
- Porcentaje de datos ausentes.
- Tipo de variable.
- Distribución de la variable.
- Presencia de outliers.
- Relación con otras variables.
- Importancia de la variable para el modelo.
- Contexto del problema de negocio.

20. El problema del Data Leakage

Uno de los errores más importantes durante la preparación de datos es utilizar información del conjunto de prueba para calcular transformaciones que deberían aprenderse únicamente con los datos de entrenamiento.

Por ejemplo, si calculamos la mediana utilizando todo el dataset antes de dividirlo en entrenamiento y prueba, estamos permitiendo que información del conjunto de prueba influya en el proceso.

Regla profesional

Primero se separan los datos en entrenamiento y prueba. Luego se calcula la estrategia de imputación utilizando solamente el conjunto de entrenamiento. Finalmente, esa transformación se aplica al conjunto de prueba.

21. Pipeline de Scikit-learn

Los pipelines permiten encadenar transformaciones y modelos de manera reproducible.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
```

```
('model', LogisticRegression())  
])
```

Este enfoque ayuda a evitar errores de preparación y permite aplicar exactamente las mismas transformaciones durante entrenamiento y predicción.

22. Errores frecuentes

- Eliminar automáticamente todas las filas con faltantes.
- Utilizar siempre la media sin analizar la distribución.
- Utilizar la moda sin considerar el equilibrio de categorías.
- Tratar códigos especiales como valores reales.
- No investigar por qué faltan los datos.
- Calcular estadísticas utilizando también el conjunto de prueba.
- Imputar antes de realizar la división entrenamiento/prueba.
- Aplicar estrategias diferentes entre entrenamiento y producción.

23. Flujo profesional para tratar valores faltantes

1. Detectar los valores faltantes.
2. Medir su cantidad y porcentaje.
3. Investigar el motivo de la ausencia.
4. Analizar su distribución.
5. Separar entrenamiento y prueba.
6. Elegir una estrategia.
7. Ajustar la estrategia con los datos de entrenamiento.
8. Aplicar la transformación.
9. Validar el resultado.
10. Documentar la decisión.

24. Ejemplo completo

Un flujo sencillo utilizando Scikit-learn puede ser:

```
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.impute import SimpleImputer  
  
df = pd.read_csv('clientes.csv')  
  
X = df.drop(columns=['abandono'])  
y = df['abandono']  
  
X_train, X_test, y_train, y_test = train_test_split(  
X, y, test_size=0.2, random_state=42  
)  
  
imputer = SimpleImputer(strategy='median')
```

```
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)
```

La diferencia entre **fit_transform()** y **transform()** es fundamental: el primero aprende los parámetros utilizando el entrenamiento y el segundo aplica esos parámetros a nuevos datos.

25. Idea central

Limpiar datos no significa borrar problemas.

Una limpieza profesional busca comprender el origen de las inconsistencias, conservar la mayor cantidad de información posible y aplicar transformaciones que puedan reproducirse correctamente en nuevos datos.

26. Glosario

Concepto	Definición
Valor faltante	Observación para la que no se dispone del valor de una variable.
Imputación	Proceso de estimar y reemplazar valores faltantes.
Media	Promedio aritmético de un conjunto de valores.
Mediana	Valor central de una distribución ordenada.
Moda	Valor que aparece con mayor frecuencia.
KNN	Método basado en vecinos cercanos para realizar estimaciones.
Data Leakage	Uso indebido de información que no debería estar disponible durante el entrenamiento.
Pipeline	Secuencia reproducible de transformaciones y pasos de procesamiento.
SimpleImputer	Herramienta de Scikit-learn para imputación de valores faltantes.
IterativeImputer	Método de imputación que utiliza relaciones entre variables.